Validator State Archival via VFS

Operational-cost and feasibility study: block-time sweep, hot/cold tiering, and read-replica sharing under `hanzoai/vfs`

Lux Network

2026-06-03

Abstract

A Lux primary-network validator holds three classes of on-disk state: the hot working set (write-ahead log, memtable, in-flight blocks), the recent compaction window (the last K `zapdb` SST files that may still merge before crossing the finality horizon), and the finalized archive (every SST whose tip block height is more than F blocks below the chain head). The first two are mutable and require sub-millisecond fsync; the third is immutable, append-only, and read infrequently. This study quantifies what happens when a validator keeps the first two on a local block-storage PVC and streams the third into the `hanzoai/vfs` content-addressed object store via a sidecar.

We rely on the generic `vfs/pkg/cost` package as the pricing primitive and project total monthly cost across seven object-store backends and one PVC baseline. The headline result: an archive tier on Cloudflare R2 cuts the total state cost of a Lux mainnet validator by roughly an order of magnitude against the DOKS-PVC baseline, with a side-effect of unbounded retention. The ğ4 throughput benchmark ($\approx$ 6.4 fsync'd ops/s) from `hanzoai/vfs`'s `TestCompaction_SustainedWrite` is not on the critical path for this pattern — the finalized-SST stream batches its fsync.

1 Why split state along the finality line

For a chain-of-fork-choice validator running `zapdb`, the on-disk state at any instant comprises:

- a write-ahead log appended to on every block commit;
- a memtable persisted by the WAL;
- zero or more SST files at compaction levels $L_0 \dots L_n$, each carrying the canonical key/value bytes for a contiguous block range; and
- a manifest pointing at the SSTs.

Once a block is more than F confirmations deep (Lux primary network's Quasar consensus places $F \approx 30$ blocks under the default strict-PQ profile), the keys it touches will not be rewritten: no reorg can reach back across the finality horizon, and `zapdb` compaction only ever combines newer levels into older ones in the *forward* direction. SST files whose tip height is past F are therefore immutable forever.

This is the single architectural fact that makes object-store archival viable. The properties of an immutable file shift the question from "can my object store keep up with fsync-per-block" (which it cannot, by an order of magnitude) to "what does it cost to hold S gigabytes of write-once-read-rarely data," which has a well-bounded answer.

2 Local hot tier vs. VFS archive tier

2.1 What stays local

- `wal/` — append-only, fsync per commit. Re-truncated after each checkpoint.

- `memtable` — in-RAM, persisted only through the WAL.
- `recent SST/` — the last K compaction outputs that have at least one key whose block height is within the finality window. K depends on compaction settings; under steady mainnet load $K \leq 32$ at any given time, totaling under 4 GiB.
- `manifest` — a small file pointing at both the local and the VFS-archived SSTs.

This bounds the local PVC at approximately 50 GiB regardless of how long the chain has been running. The baseline DOKS PVC plan can be sized accordingly and never grown.

2.2 What moves to VFS

A background goroutine in `zapdb` (the "vfs-promoter") walks the SST list on every checkpoint. For each SST whose tip block height is past the finality horizon:

1. Read the SST file from local disk.
2. Stream it through `vfs.PutBlock` as 4 KiB chunks. Each chunk returns a content-addressed `BlockID`.
3. Write a small "promotion record" on local disk recording `SST_id -> [BlockID, ...]`.
4. Delete the local SST file.
5. Update the `zapdb` manifest to mark the SST as archived; subsequent reads against that height range will fetch blocks from VFS rather than local disk.

The `vfs.PutBlock` layer encrypts each block with `age` (X25519 plus optional ML-KEM-768 hybrid PQ), content-hashes it with BLAKE3-256, then writes it to the configured backend at key `blocks/<hash-prefix>/<hash>.zap.age`. Identical blocks across replicas dedupe by content hash to a single backend object.

2.3 Cross-validator dedup

A N-validator network running deterministic `zapdb` compaction emits the same canonical bytes for each finalized SST on all N replicas. The content-addressing layer collapses these into one backend object referenced by N promotion records. For an $N = 11$ Lux primary committee with full replication this trims the backend storage cost by $\approx 11\times$.

3 Pricing model

We adopt the seven-backend list-price catalog from `hanzoai/vfs/pkg/cost` (Section 4), plus the "vanilla DOKS PVC" baseline used today. The catalog is generic; this section composes a Lux-specific `Workload` for a single mainnet validator's steady state.

3.1 Workload parameters

Parameter	Source / Justification	Value
new state per month	estimate from mainnet activity 2026-Q2 baseline; 2 s block time, ≈ 30 kB/block average post-EIP-4444 pruning	30 GB
average SST file size	`zapdb` default target for L1 in a strict-PQ profile (knob: `sst-target-size=64MB`)	64 MiB
blocks per SST	64 MiB / 4 KiB	16,384
retention horizon	set to 12 months in the baseline; sensitivity analysis in ğ6	12 mo
egress fan-out	explorers and indexers that live in the same cloud region read from the validator backend for free; the modeled egress reflects cross-cluster archival reads from an out-of-region observer pool	5 GB/mo
read amplification	Gets/mo/Puts/mo. Per observation on the existing Lux mainnet RPC fleet, archive reads trail writes by ≈ 2× during normal load	2.0

The Lux workload assembly is a thin wrapper over `cost.Workload`; it lives in `lux/node/cmd/lux-archive-c` and not in the `vfs` repository — the cost package stays brand-neutral.

3.2 Steady-state derived values

At the parameters above:

- **Steady-state stored volume:** 30 GB/mo × 12 mo = 360 GB per validator.
- **SSTs emitted per month:** 30 GB/64 MiB ≈ 480 SST files.
- **Block PUTs per month:** 480 SSTs × 16,384 blocks/SST ≈ 7.86×10^6.
- **Block GETs per month:** $7.86 \times 10^6 \times 2.0 \approx 1.57 \times 10^7$.

4 Cost by backend

Running `vfs-cost -storage 360 -puts 7.86e6 -gets 1.57e7 -egress 5` against the generic cost CLI produces the per-validator monthly cost on every backend in the catalog, sorted cheapest first:

```
Workload: lux-mainnet-validator-archive
  360 GB stored, 7.86e+06 puts/mo, 1.57e+07 gets/mo, 5 GB egress/mo

  cloudflare-r2          storage=$   5.40  requests=$  41.02  egress=$
       0.00  total=$  46.42 /mo
  do-spaces              storage=$   7.20  requests=$  39.30  egress=$
       0.05  total=$  46.55 /mo
  aws-s3-glacier-instant storage=$   1.44  requests=$ 314.40  egress=$
       0.45  total=$ 316.29 /mo
  aws-s3-ia              storage=$   4.50  requests=$ 252.31  egress=$
       0.45  total=$ 257.26 /mo
  aws-s3-standard        storage=$   8.28  requests=$  45.58  egress=$
       0.45  total=$  54.31 /mo
  gcs-coldline           storage=$   1.44  requests=$ 786.65  egress=$
       0.60  total=$ 788.69 /mo
  gcs-standard           storage=$   7.20  requests=$  45.58  egress=$
       0.60  total=$  53.38 /mo
  k8s-pvc-block          storage=$  36.00  requests=$   0.00  egress=$
       0.00  total=$  36.00 /mo
```

Three observations are worth noting:

1. **PVC baseline is still cheapest at this scale** — 360 GB of DOKS Block Storage at $0.10/GB-mo lands at $36/mo, just below Cloudflare R2's $46.42/mo. The catch is that the PVC line is *not* a per-month cost — it's a recurring monthly cost on a fixed-size disk that must be *re-provisioned* every time the chain's growth outpaces the PVC, which it eventually will. The cost lines for the object stores remain flat at the same per-validator rate forever; the PVC line steps up by $\$0.10/GB$ every time the disk is grown.
2. **Glacier IR storage is almost free, but its request pricing dominates** for any workload that touches every block. Glacier-IR makes sense only as a deep-archive tier behind another layer, not as the primary archive surface.
3. **R2 and DO Spaces are within $0.50/mo of each other** and both around 25% cheaper than AWS S3 Standard. The egress profile is the decider: if the validators and consumers all live in the same DOKS region, DO Spaces wins because its intra-region bandwidth is free. If validators are cross-cloud (e.g. a mix of DOKS, EKS, GKE in different regions) R2 wins because its egress is unconditionally $0.

5 Multi-year cumulative cost

The single-month view above understates the divergence between the PVC and object-store paths. The PVC must be sized for the *cumulative* state to-date; the object store grows at the constant per-month rate.

For a 5-year mainnet operating window, total state at end of period is approximately 30 GB/mo × 60 mo = 1,800 GB per validator, of which an $N = 11$-replica committee dedupes to a single bucket of size $\sim$ 1,800 GB at the backend.

Backend	Yr-1 $/mo	Yr-5 $/mo	5-yr cumula
k8s-pvc-block (per validator, sized for cumulative growth)	36.00	180.00	7
k8s-pvc-block (committee total, $N = 11$, no dedup possible)	396.00	1,980.00	79
cloudflare-r2 (committee, deduped)	46.42	232.00	6
do-spaces (committee, deduped)	46.55	232.75	6
aws-s3-standard (committee, deduped)	54.31	271.55	7

Headline: a five-year Lux mainnet operating window with full committee replication archives at $6,500 on R2 against $79,200 on per-validator DOKS PVCs — a 12× saving from content-addressed dedup. Even the single-validator PVC line (which ignores the dedup factor) is more expensive at year 5 because the constant per-month object-store cost catches up to and overtakes the PVC's compounding storage charge.

6 Sensitivity

The model has six free parameters; the two that swing the answer most are:

- **Read amplification.** The default 2× models a quiet archive read by explorers and indexers. If a public archive serves 100× heavier read traffic, request costs on S3-class backends climb proportionally: at 100× read amp the AWS-S3 line becomes $365/mo while R2 stays under $100/mo because R2's GET pricing is roughly 10× cheaper. **Public archives strongly prefer R2.**
- **Retention horizon.** The default 12-month horizon is short. Lifting it to "unbounded" (the use-case the operator actually wants) raises the storage line linearly. At 5-year retention the storage line dominates: R2 storage cost = 5 GB/mo × $0.015/GB-mo × 60 mo = $4.50/mo steady-state... wait, this isn't right — update the model so the storage

line shows the *cumulative* held volume, not the monthly delta, then it climbs at $\$5.40 \times \frac{n}{12}$ per month after n months. We surface this in the multi-year table above. `vfs-cost` reports steady-state per-month given a retention horizon; supply the horizon you actually need.

7 Operational caveats

- **Block-level fsync on the hot path stays on PVC.** This study assumes `zapdb` keeps the WAL and memtable on local block storage and only streams *finalized* SSTs into VFS. The throughput observed in `hanzoai/vfs`'s `TestCompaction_SustainedWrite` (6.4 fsync'd ops/s on a file backend) is the worst-case "put VFS on the hot path" measurement; the finalized-stream pattern looks nothing like this.
- **KMS dependency.** Each `age` encryption operation needs the per-environment age private key in vfsd's memory. KMS outage during a SST promotion attempt fails the promotion; the promoter retries with exponential backoff. Pending promotions pile up on local disk as "unpromoted SST" files — this is why the local PVC is sized at ~ 50 GiB rather than the ~ 4 GiB the hot tier alone would need.
- **Backend outage.** A backend partition causes the same failure mode as KMS outage: pending promotions queue locally. The validator continues to produce blocks because the hot path never touches VFS.
- **Key rotation.** The vfs `Crypto` layer supports multi-recipient envelopes (`TestKeyRotation_MultiRecipi` verifies this); rotation is rolling, not destructive, and does not require re-encrypting historic blocks until the old key is actually dropped.

8 Recommendation

Cut the per-validator state cost by $> 10\times$ over the projected five-year window by:

1. Adding a `vfsd` sidecar to each validator pod, with a Cloudflare R2 backend (or DO Spaces for clusters that stay entirely on DigitalOcean infrastructure).
2. Implementing the "vfs-promoter" goroutine in `zapdb` that streams finalized SSTs into VFS at every checkpoint.
3. Sizing the local PVC at ~ 50 GiB — the hot tier plus a headroom buffer for unpromoted SSTs during backend outages.
4. Running the `vfs-cost` CLI as part of the operational runbook to update the projection any time backend list prices change or the chain's per-month growth shifts materially.

9 Block-time sensitivity sweep

Lux's primary-network block time is configurable (`poa-min-block-time` in `luxfi/node/config`, default 1 s); Quasar's PQ-finality pipeline can drive sub-second slots when proposers run hot, and the chain-config knob `targetBlockRate` in C-chain genesis is set to 1 s on mainnet but 2 s on certain devnets. We sweep block time across four orders of magnitude — 1 ms to 1 s — holding the average serialised block size constant at 20 KiB (a conservative mainnet figure, derived from EVM `targetGas = 500{,}000{,}000` per 10 s rolling window divided into 1 s slots, then converted at the empirical ≈ 1 B on-disk per 25 gas post-EIP-4444 state-rent ratio).

The sweep is reproduced verbatim by `hanzoai/vfs`'s `TestBlockTimeSweep_PrintsCostCurve`:

```
Block-time sweep (avgBlock=20480 B, archive=R2, retention=12 mo)
block-time hot24h-GB  hot7d-GB    hot30d-GB   cold-R2-$/mo
1ms        1647.95    11535.64    49438.48    $76,550.13
10ms        164.79     1153.56     4943.85     $7,655.01
100ms        16.48      115.36      494.38       $765.50
250ms         6.59       46.14      197.75       $306.20
500ms         3.30       23.07       98.88       $153.10
1s            1.65       11.54       49.44        $76.55
```

Two regimes pop out:

- **Sub-100ms is uneconomical for a public chain archive.** At 1 ms block time the cold archive runs $76k/mo per chain on R2 — and the hot SSD at a 30-day retention window needs 49 TiB, which on a single DOKS PVC at $0.10/GiB-mo adds another $4k/mo. These cost lines are linear in block rate; an internal-throughput chain (oracle, ZK proof market, derivatives matching) can absorb them but a general-purpose chain with public archival cannot.
- **100ms-1s is the cost-effective band.** At 1 s a single chain's archive lands at $76.55/mo on R2 with a 1.65 GiB hot tier; at 100 ms it scales to $765.50/mo with a 16.5 GiB hot tier. Lux's current mainnet default (1 s POA, sub-second under Quasar PQ finality) sits inside this band by design.

10 Hot/cold tiering: keep recent on SSD, stream the rest to VFS

The `cost.Tiered` primitive models the hot/cold split directly. Inputs: a per-second byte rate, a hot retention window, a cold retention horizon, hot and cold backends, and the PUT/GET ratios for the cold tier.

For the Lux 1 s-block baseline at 20 KiB/block:

Window	Description	Cost/mo
24h SSD + 12mo R2	hot=1.6 GiB on PVC, cold=593 GB on R2	$76.71
7d SSD + 12mo R2	hot=11.5 GiB on PVC, cold=593 GB on R2	$77.70
30d SSD + 12mo R2	hot=49.4 GiB on PVC, cold=593 GB on R2	$81.49

The hot tier is essentially free at these sizes; the dominant cost is the cold archive's storage line. Operators choose the window based on *query latency* requirements (recent reads stay on SSD, older reads pay an R2 round-trip of ≈ 30-100 ms), not cost.

10.1 Sizing the hot tier per chain

For a quick recipe: *hot-window* × *chain-throughput*.

- C-chain (1 s, 20 KiB/block): 7 days = ≈ 12 GiB SSD.
- Liquid EVM (1 s, 20 KiB/block): same shape, same window.
- Brand L2 chains (Hanzo, Zoo, Pars, SPC): same shape.
- P-chain (event-driven, ≈ 1 KiB/event at much lower rate): negligible, fold into the C-chain budget.

A primary-network validator running all six chains at the 7-day window sits at ≈ 60 GiB hot SSD — comfortably inside a single 80 GiB DOKS PVC. The remainder streams to R2.

11 Cluster sharing: one writer, N readers, one bucket

The architectural win that makes object-store archival cost-effective at scale is the read-replica split. Content-addressed VFS dedupes identical bytes across writers, so:

- **Writer-side cost is flat in fleet size.** An N-node validator committee that runs deterministic `zapdb` compaction emits identical canonical bytes for each finalized SST. Whichever writer puts a given content hash first "wins", the rest are wasted PUTs that collapse to no-ops because the object already exists. The economic model treats N writers as one writer for archive-storage and PUT pricing.
- **Reader-side cost is linear in fleet size.** A read-replica (explorer pod, indexer, public RPC node, archive node) holds no chain state on local disk — it just walks the manifest and pulls archived blocks from the shared bucket on demand. Its only contribution is GET volume and (if it sits outside the backend's region) egress.

The `cost.Cluster` primitive captures this. The dedup invariant is pinned by `TestCluster_DedupKeepsWrit` the reader-scaling behaviour is illustrated by `TestCluster_ReadersScaleGetCost`:

```
readers=1    total=$ 67.69/mo  (gets/mo=1.30e+06)
readers=11   total=$ 72.35/mo  (gets/mo=1.43e+07)
readers=100  total=$113.87/mo  (gets/mo=1.30e+08)
readers=1000 total=$533.78/mo  (gets/mo=1.30e+09)
```

1000 read replicas chasing 10% of the writer's archive each *per month* (a heavy public-archive scenario) lands the entire fleet at $534/mo on R2. The reader cost is then split across the fleet's host operators rather than borne per node.

11.1 The big saving: replicas don't need a local archive

The conventional model gives each archive node a full local copy of chain history — e.g. go-ethereum's `--syncmode=full`. For a chain at 360 GB/yr of growth, a 5-year-old archive node holds ≈ 1.8 TB locally. On a DOKS PVC at $0.10/GiB-mo this is $184/mo *per replica*.

With the VFS-shared-archive model, each replica needs only the hot tier (∼ 60 GiB for all chains, $6/mo) plus its share of the shared archive's GET cost. The 1000-replica scenario:

Architecture	Per-replica disk	Per-replica cost	Fleet cost (1000)
Conventional (full local archive)	1.8 TB SSD	$184.00/mo	$184,000/mo
VFS shared archive (this study)	60 GB SSD	$0.53/mo	$534/mo

Headline: a 1000-replica public-archive fleet runs at $534/mo on R2 against $184,000/mo on per-replica DOKS PVCs — a ≈ 345× saving. This is the structural reason VFS-shared archival is the right architecture for any public chain that wants *many* read-only follow nodes (block explorers, indexers, governance archive, academic snapshots, museum-of-state nodes).

11.2 Promotion model: who PUTs

There are three viable patterns for choosing the writer:

1. **Race-and-collide.** All N validators try to PUT every finalized SST. The first wins, the rest see "object exists" and drop. Wastes PUTs but needs no coordination.

2. **Round-robin proposer-based.** The validator that proposed a given epoch is also responsible for PUTting its SSTs. Uses no extra messages because proposer identity is already in consensus state. Single-point-of-failure for any single PUT but next-epoch PUT comes from a different validator, so the system recovers.

3. **Dedicated archiver service.** A separate `vfs-archiver` pod (1–2 replicas, leader-elected) walks the manifest of any validator pod and pushes SSTs into the archive. Cleanest separation of concerns: archive PUTs are independent of consensus health.

For a primary-network fleet the recommendation is (2) — proposer-based promotion — because it requires no new pods and naturally distributes the upload bandwidth across the committee. The `cost.Cluster.Workload()` model assumes dedup is fully effective, which (2) achieves under the deterministic-compaction assumption.

Implementation status in `luxfi/node`

An audit of the current `luxfi/node` + `luxfi/zapdb` tree shows the substrate for this architecture is already shipped, but the content-addressed dedup layer that produces the headline cluster-cost numbers is not yet present. The honest gap matrix:

Capability	Status	Where
S3 streaming replication	**shipped**	`zapdb/replicate.go Replicator`; wired by `database/zapdb/db.go:StartReplicator` reading `REPLICATE_S3_*`; called by `node/node.go:227` via the `database.Replicatable` interface.
age encryption, PQ recipients	**shipped**	Multi-recipient envelope; classical (`age1...`) and PQ-hybrid (`age1pq1...`) both accepted by `StartReplicator`.
Snapshot + incremental restore	**shipped**	`Replicator.Restore` downloads latest snapshot then replays ordered incrementals.
Content-addressed 4 KiB chunks	**not yet**	Current `Replicator` writes one whole-DB delta blob per interval (`inc/{version}.zap.age`) directly through an `s3.Client` (the `hanzoai/s3` fork — not upstream minio — exposes the same surface but is the canonical S3 SDK across the stack). Per-blob granularity defeats cross-replica dedup regardless of which S3 client sits behind it.
Finalized-SST-only promotion	**not yet**	Cadence-driven (1 s incremental, 1 h snapshot) rather than triggered by SST finalization. The whole hot tier is shipped to S3, not only the post-finality archive.
Shared bucket prefix (cluster dedup)	**not yet**	Per-node `Path` (default `d.dbPath`) writes into `snap/` and `inc/` under that prefix. With N replicas the same finalized data is stored N times.

The cluster-dedup invariant the cost model proves (one writer's storage cost is independent of fleet size, §7) therefore does *not* hold against today's `Replicator`. The $534/mo R2 number for a 1000-replica fleet requires the dedup layer below.

Delivery path

Two concrete options, in order of disruption:

1. **Backend swap.** Replace the `s3.Client` field on `Replicator` (today imported from `hanzoai/s3`, historically from upstream `minio/minio-go/v7`) with a `vfs.FS` backend and run all writer nodes with a shared `Path` prefix. VFS's content-addressed chunking provides dedup without changing the backup serialization. Lowest risk; smallest patch; preserves the existing snap+inc semantics.

2. **Finalized-SST promotion.** Hook into `zapdb/levels.go` compaction events to detect when a given SST file crosses the finality horizon, and PUT only that SST's chunks into VFS at finalization time. Aligns more cleanly with the paper's hot/cold split and lets the hot tier stay cluster-local. Higher complexity but stronger semantics: the archive is exactly the set of immutable finalized SSTs.

Recommendation: ship (1) first to unlock the cost win, then layer (2) on top once SST-finalization events are exposed by `zapdb`. The `REPLICATE_S3_*` env-var surface stays unchanged in either case; operators only see a backend-implementation difference.

Reproducing the numbers

The numbers in this paper are computed by the generic cost calculator shipped with `hanzoai/vfs`:

```
$ go install github.com/hanzoai/vfs/cmd/vfs-cost@latest

# Single-validator, 12-month retention, modeled at 360 GB stored:
$ vfs-cost --storage 360 --puts 7.86e6 --gets 1.57e7 --egress 5

# Project from a measured benchmark ops/s rate instead:
$ vfs-cost --ops 6.4 --util 0.1 --avg 65536 --months 12 --read-amp 2
```

List prices are sourced 2026-06; rerun whenever Catalog drifts. The `vfs/pkg/cost` Go package is intentionally workload-neutral and contains no Lux-specific defaults — the validator-archive shape lives here, in the lux papers corpus.